Structure-based bioinformatics with BiochemicalAlgorithms.jl

Jennifer Leclaire[1], Thomas Kemmer[1], and Andreas Hildebrandt[1]

[1]Scientific Computing and Bioinformatics, Institute of Computer Science, Johannes Gutenberg-University Mainz

ABSTRACT

Keywords

Julia, Structure-based bioinformatics, C++, BALL

1. Introduction

The aim of structural bioinformatics is the analysis and targeted manipulation of three dimensional structures of biological macromolecules such as proteins and nucleic acids (RNA/DNA). This research area combines and applies knowledge from diverse disciplines ranging from fundamental physical laws such as Newton's equation of motion to complex situations requiring in-depth knowledge of biochemistry and advanced numerical computing methodologies.
More precisely, molecular modeling techniques typically rely on molecular mechanics in which a molecular force fields is being used to compute the energy of the examining structure. Resulting applications include structure minimization, molecular dynamics (MD) simulations and molecular docking scenarios. The latter is inevitable for drug design because it computes the best configuration for a given protein and a ligand (the drug) and compute the binding affinity. The importance of molecular applications has been demonstrated during the Covid-19 pandemic. Tools for molecular modeling and in particular docking suites were attracting widespread interest again in order to model SARS-COV-2 proteins and running molecular docking simulations.

Write about: Rational Drug Design economically important

Why do we have to run simulations on the computer? Why is it necessary to perform it on a computer? Example Protein Ligand Docking... As an example.
For more than 50 years researcher have been studying molecular functions of proteins. The first attempts date back to 1959, when Max Perutz and John Kendrew used X-ray crystallography to to solve the structure of hemoglobin.
A vital aspect of this research field is the availability of molecular structure data, which was solved by the initialization of the protein data bank in 1972. Although it only contained 7 structures, it now provides 227.561 experimentally solved structures (dated on:). With the rapid rise of computed structures

check number of structures in pdb

software development in interdisciplinary research areas such as structural bioinformatics was, and still is, typically challenging. Software packages for handling molecular structures and molecular applications are available for many years and can be rougly divided into open-source and closed-sourced tools. Most software packages were created from 1995 to 2010 and written in C++. Although this choice of programming language enables the required efficiency, it does not allow for rapid prototyping, which is why some software packages provide an additional interface in a scripting language like Python.

Cite

The tools were often only designed for one specific task e.g., implementation of a structure minimization algorithm, docking algorithm... One example for the latter is Schroedinger

cite single approache

—docking algorithmus

—kraftfelder..

The development of software packages for structural bioinformatics remains a challenging task.

cite schroed

Several packages already exist in Julia related to structural bioinformatics. Most prominently, the packages under the two Github communities *Molecular Simulation in Julia* and *BioJulia*, which puts an emphasis on sequential bioinformatics.
Molly.jl is an excellent package for molecular simulations written in Julia and is part of the *Molecular Simulation in Julia* Github community. Additionally, *ProtoSyn.jl* is an interesting approach to handle and manipulate oligopeptides but does not seem to be actively maintained any more (the last push is 10 months ago).
A platform from which molecular file formats can be read and write, the entire preprocessing pipeline can be integrated and the infrastructure for molecular mechanics are provided is still lacking. There remains a need for a basis from which software packages for handling molecular file formats and the proper preprocessing
Here, we present BiochemicalAlgorithms.jl . We provide the basis for interesting analysis encompassing the entire molecular modeling pipeline:

—Reading common data formats such as PDB, hin, mol and JSON

—Preprocessing the input by preparing the entire system ready to simulate.

—Molecular Mechanics such as AMBER ForceField

—(Output writing) such as JSON

BiochemicalAlgorithms.jl is designed to be a platform from which other packages can be included.

2. BALL - Biochemical Algorithms Library

As already mentioned, BiochemicalAlgorithms.jl is based on the BALL framework. The main intention for the development of BALL as well as for BiochemicalAlgorithms.jl is to generate a framework for rapid prototyping of molecular applications. This section summarizes the key concepts of BALL that motivated the

design of our project [1]
The initial work on the BALL project started in 1996 resulting in the C++ written library BALL and its accompanying molecular viewer *BALLView* [**?**]. The development followed an object-oriented design and was focused on four design goals: *ease of use*, *openness*, *robustness* and *functionality*. Hildebrandt et al published an updated version in 2010 featuring among other things CMake-Buildsystem and Python-Bindings.
One reason for the success of BALL is its sophisticated design: As can be seen in Figure 1 it is structured in several layers. On top of the standard template library (STL), resides the foundation classes providing a set of general data structures such as hash sets and mathematical objects e.g. matrices, vectors,... . The third layer is the actual core of the library: the `KERNEL` classes, which contains data structures for molecular entities. The basic components represent basic functionalities and are placed on top of the core with the exception of the visualization module that is based on QT and Open GL. Finally, the application layers can be used to develop own applications or use the available tools. Bindings in Python were created for the purpose of increasing usability without the necessity of interacting with the C++ kernel.
The latter is shown in Figure 2 and its classes are forming three different frameworks: the general molecular framework, the protein and the nucleic acid frameworks. All kernel classes are implemented through the realization of a composite pattern. More precisely, the composite class is the base class for all derived classes representing molecular entities such as `Atom`, `Protein`, etc. or container classes `System`, `AtomContainer`, and so on.
Taken together BALL is a very well designed and structured software package, which is also reflected by its long lasting popularity. It used to have one of the biggest user community for an open-source software in this area.

3. A fresh approach for Structural Bioinformatics: BiochemicalAlgorithms.jl

In this work, we sought to redesign the popular BALL framework for molecular analysis and simulation. In this section, we first present the reason for a redesign, followed by a description of the core implementation of BiochemicalAlgorithms.jl and the internal representation closing with the benchmark results.

3.1 Why Julia?

The previous section briefly gives an overview of the main design principles of BALL . The accomplishment of the design principles are highly dependent on the choice of the programming language. Back in 1996 and 2010, C++ was the best choice for the given requirements: *ease of use*, *openness*, *robustness* and *functionality*. However, the development times play a crucial role for the acceptance and usability of the underlying software package. Nowadays, C++ is still a high performace programming language but it takes a long time to actually learn this language.

—portability

—development times

—usability

—boilerplate code

The goals can be achieved much easier ..

[1] An in-depth description of the entire BALL framwork is beyond the scope of this article. Confer the main publications [**?**, **?**] for more details.

3.2 The core

3.3 The internal representation

When working with structural data of molecular entities such as proteins, or nucleic acid molecules (DNA/RNA), the representation of three dimensional coordinates play an important role. Just like in BALL we define a similar hierarchical structure...as can be seen in fig.. Optimized for random access to any atoms... same hand easy to use for Julia users or Python users familiar with Pandas DataFrames... So initially, representation was based on DataFrames. While theis is extremely intuitive, it has some

3.4 Benchmarks

On par with its C++ predecessor it is intuitiv to write code that is

4. Applications

4.1 A trivial comparison between C++ and Julia

A trivial comparison between C++ and Julia

4.2 Water molecule

As shown in the code example Code listing 1 the interaction with core components of BiochemicalAlgorithms.jl is straightforward.

Code 1: Intuitive usage of BiochemicalAlgorithms.jl core components

```julia
using BiochemicalAlgorithms
using BiochemicalVisualization

sys = System()
h2o = Molecule(sys)

o1 = Atom(h2o, 1, Elements.O)
h1 = Atom(h2o, 2, Elements.H)
h2 = Atom(h2o, 3, Elements.H)

h1.r = [1, 0, 0]
h2.r = [cos(deg2rad(105)), sin(deg2rad(105)), 0]

Bond(h2o, o1.idx, h1.idx, BondOrder.Single)
Bond(h2o, o1.idx, h2.idx, BondOrder.Single)

println("Number of atoms: ", natoms(h2o))
println("Number of bonds: ", nbonds(h2o))

ball_and_stick(sys)
stick(sys)
van_der_waals(sys)
```

4.3 RMSD computation and Application of Amber

Code 2: Comparison and mapping of two similar structures

```julia
sys = load_pdb.(["data/arnd1.pdb",
                 "data/arnd2.pdb"])

fdb = FragmentDB()
normalize_names!.(sys, Ref(fdb))
reconstruct_fragments!.(sys, Ref(fdb))
build_bonds!.(sys, Ref(fdb))

println.(sys)

compute_energy.(AmberFF.(sys), verbose=true)
```

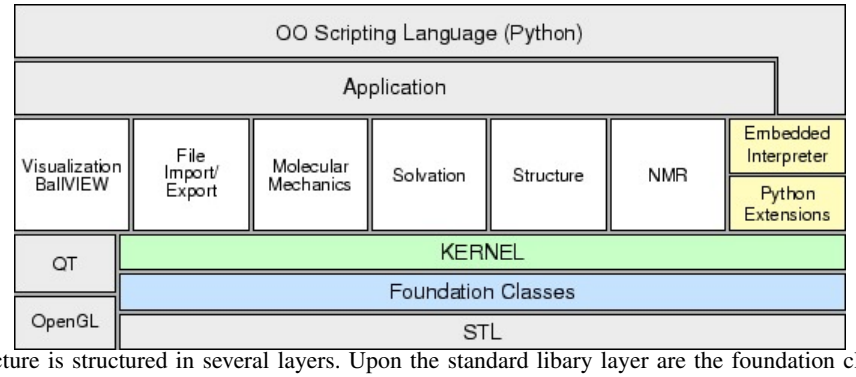

Fig. 1: BALL 's architecture is structured in several layers. Upon the standard libary layer are the foundation classes and ontop of them the kernel. Several module extend the interface for visualization, file import and expoert, molecular mechanics, solvation, structure and NMR. The C++ written frameworkis extended by Python interface for fast scripting purposes. The figure was taken from the official BALL documentation [**?**].

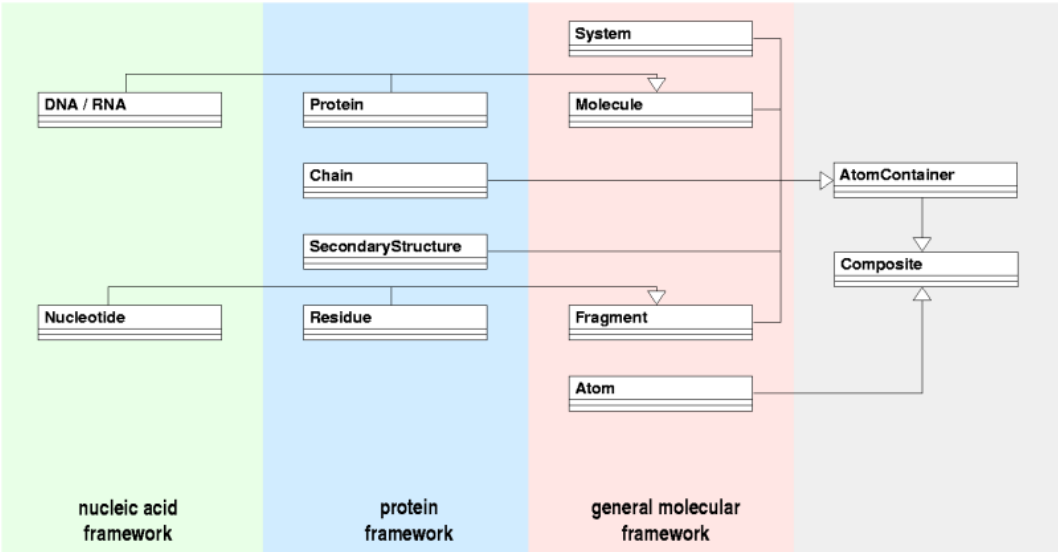

Fig. 2: UML class diagram of the kernel classes. The kernel classes form three frameworks and are implemented using the composite pattern. The figure was taken from the official BALL documentation [**?**].

```
12
13 println("RMSD before mapping: ",
14         compute_rmsd(sys[1], sys[2]))
15
16 map_rigid!(sys[1], sys[2])
17
18 println("RMSD after mapping: ",
19         compute_rmsd(sys[1], sys[2]))
```

4.4 Visualization using BiochemicalVisualization.jl

A key feature of BiochemicalAlgorithms.jl is the visualization tool *BiochemicalVisualization.jl* that has been developed alongside the main framework. As shown in Figure 4 BiochemicalVisualization.jl currently supports three different representation of protein structures, namely `ball-and-stick`, `van-der-Waals` and `stick` (cf. code listing 1).
When dealing with three dimensional structures of macromolecules, visualization plays an important role for supporting the development of insights into molecular functions. The possibility to visualize and interactively modify the representations provides great support during modelling scenarios. For instance, the tool has been used to visualize different steps from code listing 2. The first image left represent the raw input read from the underlying pdb file, the image in the middle shows the same input after preprocession it with the fragment data base (lines4-7). Finally, the mapping of both structures is shown in the image on the left. As can be seen, the structures do not match perfectly onto each other.
Even this rather simple example already demonstrates the advantage of a visual representation that can be modified and manipulated in context of modelling scenarios and how the visualization supports the development of knowledge of molecular functions.

5. Conclusion

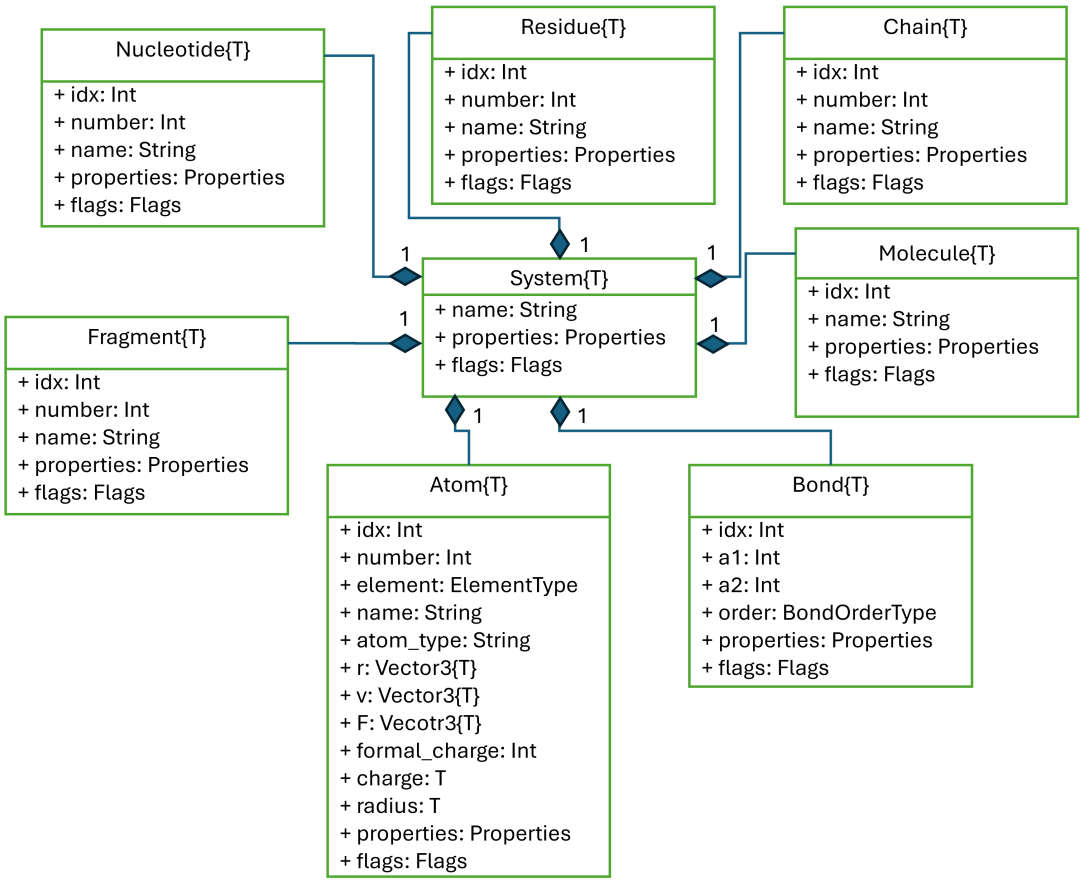

Fig. 3: UML-Diagramm of the core of BiochemicalAlgorithms.jl . In the center resides the `System` interface. All other functionalities are grouped around that core piece. Only the most important functionalities of each class are shown.

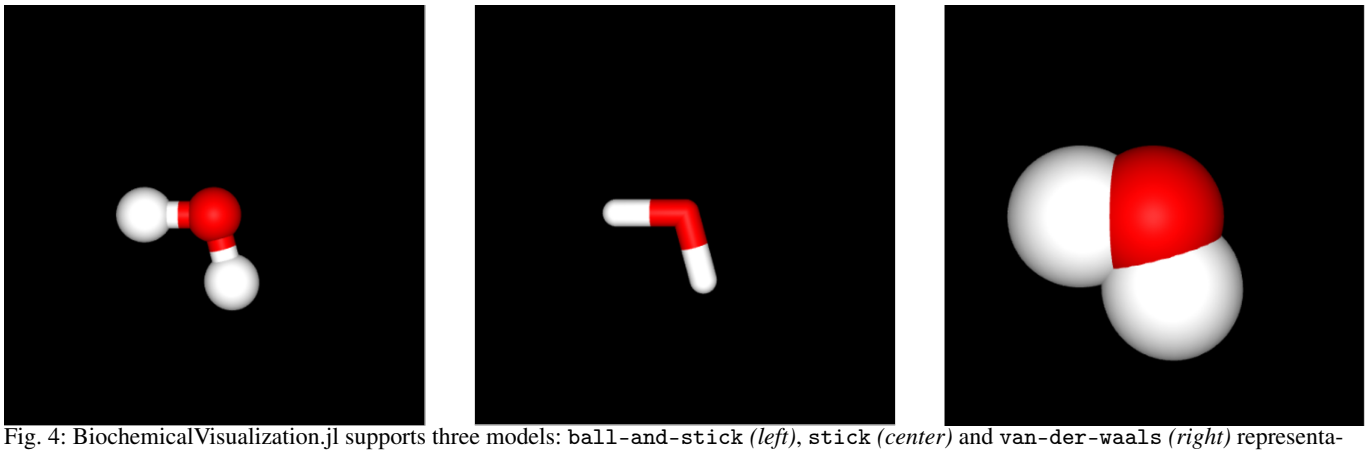

Fig. 4: BiochemicalVisualization.jl supports three models: `ball-and-stick` *(left)*, `stick` *(center)* and `van-der-waals` *(right)* representation of the water molecule as generated by the code listing 1.

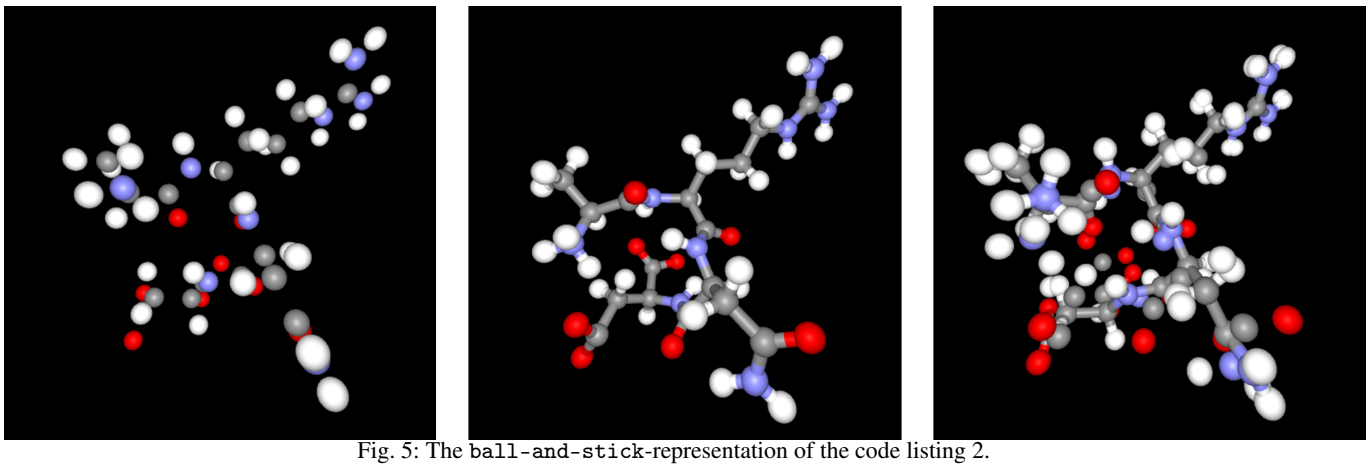

Fig. 5: The `ball-and-stick`-representation of the code listing 2.